

Suseciones

Matemática Discreta y Algoritmos - Guía de Estudio Pomodoro

Referencia Base: Rosen, Kenneth H. - Matemática Discreta y sus Aplicaciones

POMODORO 1: FUNDAMENTOS - Sucesiones Aritméticas y Geométricas (0-25 min)

Resumen Conceptual

Las **sucesiones** son secuencias ordenadas de números que siguen patrones específicos. En Matemática Discreta, los dos tipos fundamentales son:

1. Sucesiones Aritméticas (SA)

Una sucesión es aritmética si la diferencia entre términos consecutivos es constante.

Fórmula General:

$$a_n = a_1 + (n - 1)d$$

Donde:

- a_n = enésimo término
- a_1 = primer término
- d = diferencia común (razón)
- n = posición del término

Ejemplo:

- Sucesión: 2, 5, 8, 11, 14, ...
- $a_1 = 2$, $d = 3$
- $a_5 = 2 + (5 - 1) \times 3 = 2 + 12 = 14 \checkmark$

Suma de n términos:

$$S_n = \frac{n}{2}(a_1 + a_n) \quad \text{o} \quad S_n = \frac{n}{2}(2a_1 + (n - 1)d)$$

Ejemplo: Suma de los primeros 5 términos:

$$S_5 = \frac{5}{2}(2 + 14) = \frac{5}{2} \times 16 = 40$$

2. Sucesiones Geométricas (SG)

Una sucesión es geométrica si la razón entre términos consecutivos es constante.

Fórmula General:

$$a_n = a_1 \times r^{n-1}$$

Donde:

- a_n = enésimo término
- a_1 = primer término
- r = razón común
- n = posición del término

Ejemplo:

- Sucesión: 3, 6, 12, 24, 48, ...
- $a_1 = 3, r = 2$
- $a_5 = 3 \times 2^{5-1} = 3 \times 16 = 48 \checkmark$

Suma de n términos:

$$S_n = a_1 \times \frac{r^n - 1}{r - 1} \quad (\text{si } r \neq 1)$$

Ejemplo: Suma de los primeros 5 términos:

$$S_5 = 3 \times \frac{2^5 - 1}{2 - 1} = 3 \times 31 = 93$$



Comparación: Crecimiento Lineal vs. Exponencial

Aspecto	Crecimiento Lineal (SA)	Crecimiento Exponencial (SG)
Fórmula	$a_n = a_1 + (n - 1)d$	$a_n = a_1 \times r^{n-1}$
Diferencia	Constante	Multiplicativa
Velocidad	Lenta, predecible	Rápida, acelera
Ejemplo Real	Ahorros mensuales fijos	Interés compuesto
$n=10, a_1 = 1$	$1 + 9 = 10$	$2^9 = 512$

Visualización Intuitiva:

- Lineal:** Subes escaleras (cada paso suma lo mismo)

- **Exponencial:** Saltas por pisos (cada salto multiplica)
-

✓ AUTOEVALUACIÓN RÁPIDA - Pomodoro 1

Pregunta 1: En la sucesión aritmética 5, 11, 17, 23, ...

- ¿Cuál es el valor de d ?
- ¿Cuál es a_{10} ?
- **Respuesta esperada:** $d = 6$, $a_{10} = 5 + 9 \times 6 = 59$

Pregunta 2: ¿Cuándo crece más rápido: una sucesión con $d = 100$ o una con $r = 1.5$?

- **Respuesta esperada:** Inicialmente $d = 100$, pero a partir de cierto n , $r = 1.5$ (exponencial supera lineal)
 - Cálculo: $1.5^{15} \approx 437$ vs $100 \times 15 = 1500$ (todavía lineal), pero $1.5^{50} \approx 88,000$ vs $100 \times 50 = 5000$
-

🕒 POMODORO 2: RECURSIVIDAD - De la Fórmula Recursiva a la Cerrada (25-50 min)

📖 Concepto de Recursividad

Una **relación de recurrencia** define cada término basado en términos anteriores. El objetivo es convertirla en una **fórmula cerrada** (que expresa a_n directamente sin referencia a términos previos).

🔍 Método: "Desenrollar" la Recurrencia

Ejemplo 1: Sucesión Aritmética Recursiva

Definición recursiva:

$$\begin{aligned}a_1 &= 2 \\a_n &= a_{n-1} + 3 \quad \text{para } n \geq 2\end{aligned}$$

Desenrollar:

$$\begin{aligned}a_2 &= a_1 + 3 = 2 + 3 \\a_3 &= a_2 + 3 = (2 + 3) + 3 = 2 + 2(3) \\a_4 &= a_3 + 3 = (2 + 2(3)) + 3 = 2 + 3(3)\end{aligned}$$

$$a_n = 2 + (n - 1)(3)$$

Fórmula cerrada:

$$a_n = 2 + 3(n - 1) = 3n - 1$$

Verificación:

- $a_1 = 3(1) - 1 = 2 \checkmark$
 - $a_5 = 3(5) - 1 = 14 \checkmark$
-

Ejemplo 2: Sucesión Geométrica Recursiva

Definición recursiva:

$$\begin{aligned} b_1 &= 5 \\ b_n &= 2 \times b_{n-1} \quad \text{para } n \geq 2 \end{aligned}$$

Desenrollar:

$$\begin{aligned} b_2 &= 2 \times b_1 = 2 \times 5 \\ b_3 &= 2 \times b_2 = 2 \times (2 \times 5) = 2^2 \times 5 \\ b_4 &= 2 \times b_3 = 2 \times (2^2 \times 5) = 2^3 \times 5 \\ b_n &= 2^{n-1} \times 5 \end{aligned}$$

Fórmula cerrada:

$$b_n = 5 \times 2^{n-1}$$

Verificación:

- $b_1 = 5 \times 2^0 = 5 \checkmark$
 - $b_3 = 5 \times 2^2 = 20 \checkmark$
-

Ejemplo 3: Más Complejo - Fibonacci

Definición recursiva:

$$\begin{aligned} F_1 &= 1, \quad F_2 = 1 \\ F_n &= F_{n-1} + F_{n-2} \quad \text{para } n \geq 3 \end{aligned}$$

Valores iniciales: 1, 1, 2, 3, 5, 8, 13, 21, ...

Fórmula de Binet (cerrada):

$$F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}}$$

Donde:

- $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$ (razón áurea)
- $\psi = \frac{1-\sqrt{5}}{2} \approx -0.618$

Verificación:

- $F_5 = \frac{1.618^5 - (-0.618)^5}{\sqrt{5}} \approx 5 \checkmark$

Pasos Sistemáticos para Desenrollar

1. **Escribe los primeros 3-4 términos** usando la definición recursiva
2. **Identifica el patrón** en cómo se expresa cada término
3. **Factoriza** términos comunes
4. **Generaliza** para a_n
5. **Verifica** con casos específicos

AUTOEVALUACIÓN RÁPIDA - Pomodoro 2

Pregunta 1: Dada la recurrencia $c_1 = 10$, $c_n = c_{n-1} + 5$

- Desenrolla hasta c_n
- **Respuesta esperada:** $c_n = 10 + 5(n - 1) = 5n + 5$

Pregunta 2: Dada $d_1 = 1$, $d_n = 3 \times d_{n-1}$

- ¿Cuál es d_n en forma cerrada?
- **Respuesta esperada:** $d_n = 3^{n-1}$

POMODORO 3: INDUCCIÓN MATEMÁTICA - Demostraciones Rigurosas (50-75 min)

¿Qué es la Inducción Matemática?

La **inducción matemática** es una técnica de demostración para probar que una proposición es verdadera para todos los números naturales. Funciona como dominós: si el primero cae (caso base) y cada dominó hace caer el siguiente (paso inductivo), todos caen.

Estructura de una Demostración por Inducción

Para demostrar $P(n)$ es verdadera para todo $n \geq 1$:

1. BASE ($n = 1$):
Demuestra que $P(1)$ es verdadera
2. HIPÓTESIS INDUCTIVA:
Asume que $P(k)$ es verdadera para algún $k \geq 1$
(No necesitas probar esto, solo asúmelo)
3. TESIS (Paso Inductivo):
Demuestra que $P(k+1)$ es verdadera
Usa la hipótesis del paso 2
4. CONCLUSIÓN:
Por inducción, $P(n)$ es verdadera para todo $n \geq 1$

Ejemplo 1: Sumatoria de Números Naturales

Proposición: Demostrar que $\sum_{i=1}^n i = \frac{n(n+1)}{2}$ para todo $n \geq 1$

PASO 1: BASE ($n = 1$)

Lado izquierdo: 1

$$\text{Lado derecho: } \frac{1(1+1)}{2} = \frac{1 \times 2}{2} = 1 \quad \checkmark$$

$P(1)$ es verdadera

PASO 2: HIPÓTESIS INDUCTIVA

Asume que $P(k)$ es verdadera:

$$\sum_{i=1}^k i = \frac{k(k+1)}{2}$$

PASO 3: TESIS - Demuestra $P(k+1)$

$$\text{Debes demostrar: } \sum_{i=1}^{k+1} i = \frac{(k+1)(k+2)}{2}$$

Desarrollo:

$$\begin{aligned}
\sum_{i=1}^{k+1} i &= \left[\sum_{i=1}^k i \right] + (k+1) \\
&= \frac{k(k+1)}{2} + (k+1) \quad (\text{Usando hipótesis inductiva}) \\
&= (k+1) \left[\frac{k}{2} + 1 \right] \quad (\text{Factorizamos } (k+1)) \\
&= (k+1) \left[\frac{k+2}{2} \right] \quad (\text{Sumamos fracciones}) \\
&= \frac{(k+1)(k+2)}{2} \quad \checkmark
\end{aligned}$$

PASO 4: CONCLUSIÓN

Por el principio de inducción matemática,

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \text{ es verdadera para todo } n \geq 1$$

Ejemplo 2: Desigualdad con Potencias

Proposición: Demostrar que $2^n > n^2$ para todo $n \geq 5$

PASO 1: BASE (n = 5)

$$\text{Lado izquierdo: } 2^5 = 32$$

$$\text{Lado derecho: } 5^2 = 25$$

$$32 > 25 \quad \checkmark$$

$P(5)$ es verdadera

PASO 2: HIPÓTESIS INDUCTIVA

Asume: $2^k > k^2$ para algún $k \geq 5$

PASO 3: TESIS - Demuestra $P(k+1)$

Debes demostrar: $2^{k+1} > (k+1)^2$

Desarrollo:

$$2^{k+1} = 2 \times 2^k$$

$$> 2 \times k^2 \quad (\text{Usando hipótesis inductiva})$$

$$= 2k^2$$

Necesitamos mostrar que $2k^2 > (k+1)^2 = k^2 + 2k + 1$

$$2k^2 > k^2 + 2k + 1$$

$$k^2 > 2k + 1$$

$$k^2 - 2k - 1 > 0$$

Para $k \geq 5$: $25 - 10 - 1 = 14 > 0 \checkmark$

Por lo tanto: $2^{k+1} > (k+1)^2$

PASO 4: CONCLUSIÓN

Por inducción, $2^n > n^2$ para todo $n \geq 5$

Ejemplo 3: Sumatoria Geométrica

Proposición: Demostrar que $\sum_{i=0}^{n-1} r^i = \frac{r^n - 1}{r - 1}$ para $r \neq 1, n \geq 1$

PASO 1: BASE (n = 1)

Lado izquierdo: 1

$$\text{Lado derecho: } \frac{r^1 - 1}{r - 1} = \frac{r - 1}{r - 1} = 1 \quad \checkmark$$

PASO 2: HIPÓTESIS INDUCTIVA

$$\text{Asume: } \sum_{i=0}^{k-1} r^i = \frac{r^k - 1}{r - 1}$$

PASO 3: TESIS

$$\text{Debes demostrar: } \sum_{i=0}^k r^i = \frac{r^{k+1} - 1}{r - 1}$$

Desarrollo:

$$\begin{aligned} \sum_{i=0}^k r^i &= \left[\sum_{i=0}^{k-1} r^i \right] + r^k \\ &= \frac{r^k - 1}{r - 1} + r^k \\ &= \frac{(r^k - 1) + r^k(r - 1)}{r - 1} \\ &= \frac{r^k - 1 + r^{k+1} - r^k}{r - 1} \\ &= \frac{r^{k+1} - 1}{r - 1} \quad \checkmark \end{aligned}$$

PASO 4: CONCLUSIÓN

Por inducción, la fórmula es verdadera para todo $n \geq 1$

✅ AUTOEVALUACIÓN RÁPIDA - Pomodoro 3

Pregunta 1: Demuestra por inducción que $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$

Respuesta esperada (esquema):

Base: $n=1$: $1 = 1(2)(3)/6 = 1 \checkmark$

Paso: Si $\sum_{i=1}^k i^2 = k(k+1)(2k+1)/6$, entonces
 $\sum_{i=1}^{k+1} i^2 = [k(k+1)(2k+1)/6] + (k+1)^2$
 $= (k+1)[k(2k+1) + 6(k+1)]/6$
 $= (k+1)[2k^2 + 7k + 6]/6$
 $= (k+1)(k+2)(2k+3)/6 \checkmark$

Pregunta 2: ¿Por qué necesitamos AMBOS la base y el paso inductivo?

- **Respuesta esperada:** La base inicia la cadena (primer dominó), el paso inductivo la propaga (cada dominó tira el siguiente). Sin ambos, la prueba es incompleta.

🕒 POMODORO 4: APLICACIONES - Del Teórico al Mundo Real (75-100 min)

📖 Conexiones Fundamentales

Las sucesiones, recursividad e inducción no son solo abstractas: ¡gobiernan algoritmos, estructuras de datos y fenómenos naturales!

🌳 1. ÁRBOLES BINARIOS y Complejidad

Concepto:

Un árbol binario es una estructura donde cada nodo tiene 0, 1, o 2 hijos.

Relación Recursiva:

Nivel 0 (raíz): 1 nodo = 2^0

Nivel 1 : 2 nodos = 2^1

Nivel 2 : 4 nodos = 2^2

⋮

Nivel n : 2^n nodos

Total en árbol de altura h :

$$T(h) = 1 + 2 + 4 + \dots + 2^h = 2^{h+1} - 1$$

Ejemplo Práctico:

- Árbol de altura 3: máximo $2^4 - 1 = 15$ nodos
- Árbol de altura 10: máximo 2047 nodos
- Árbol de altura 20: máximo 2,097,151 nodos ← ¡Crecimiento exponencial!

Aplicación: Búsqueda binaria en estructuras de datos tiene complejidad $O(\log n)$ porque necesita altura $\log n$.

2. COMPLEJIDAD ALGORÍTMICA (Big O)

Notación Big O - Ordenes Comunes:

Notación	Nombre	Ejemplo	n=10	n=100	n=1000
$O(1)$	Constante	Acceso a array	1	1	1
$O(\log n)$	Logarítmica	Búsqueda binaria	3.3	6.6	10
$O(n)$	Lineal	Búsqueda simple	10	100	1000
$O(n \log n)$	Linearítmica	Merge sort	33	664	10,000
$O(n^2)$	Cuadrática	Bubble sort	100	10,000	1,000,000
$O(2^n)$	Exponencial	Fibonacci naive	1024	∞	∞

¿Por qué importa?

Una app con $O(n^2)$ que maneja 1000 usuarios:

- Operaciones: 1,000,000 → ~1 segundo ✓
- Con 10,000 usuarios: 100,000,000 → ~100 segundos ✗
- Con 100,000 usuarios: 10 billones → Colapsa ✗

La misma app con $O(n \log n)$:

- 10,000 usuarios: 132,877 ops → Casi instantáneo ✓
- 100,000 usuarios: 1,660,964 ops → Sigue siendo rápido ✓

Analizar Complejidad con Sumas:

Algoritmo bubble sort (dos bucles anidados):

```
for i = 1 to n:  
    for j = 1 to n:  
        comparar y intercambiar
```

Comparaciones totales: $\sum_{i=1}^n n = n \times n = n^2$

Complejidad: $O(n^2)$

Algoritmo búsqueda binaria (reducir por mitad):

Nivel 1: n elementos

Nivel 2: n/2 elementos

Nivel 3: n/4 elementos

...

Niveles totales: $\log_2(n)$

Complejidad: $O(\log n)$

3. PROBLEMAS DE LA VIDA REAL

A) Crecimiento de Usuarios en una App

Escenario: Tu startup tiene 100 usuarios iniciales. Cada mes crece 20%.

Modelo:

$$U_n = 100 \times 1.2^{n-1}$$

Proyección:

- Mes 1: 100 usuarios
- Mes 6: $100 \times 1.2^5 \approx 249$ usuarios
- Mes 12: $100 \times 1.2^{11} \approx 866$ usuarios
- Mes 24: $100 \times 1.2^{23} \approx 7,304$ usuarios
- Mes 36: $100 \times 1.2^{35} \approx 61,917$ usuarios ← Exponencial

Implicación: Necesitas infraestructura que escale exponencialmente o colapsará.

B) Proliferación de Bacterias

Escenario: Una bacteria se divide cada 20 minutos.

Modelo:

$$B_n = B_0 \times 2^n$$

Cálculo:

- $t = 0$ min: 1 bacteria
- $t = 20$ min: 2 bacterias
- $t = 40$ min: 4 bacterias
- $t = 3$ horas: $2^9 = 512$ bacterias
- $t = 8$ horas: $2^{24} = 16.7$ millones
- $t = 24$ horas: $2^{72} \approx 4.7 \times 10^{21}$ bacterias (¡más que granos de arena!)

Insight: Esto explica por qué los antibióticos deben actuar rápidamente.

C) Interés Compuesto

Escenario: Inviertes \$1000 a 5% anual compuesto mensualmente.

Modelo (Recursiva):

$$A_0 = 1000$$
$$A_n = A_{n-1} \times \left(1 + \frac{0.05}{12}\right)$$

Modelo (Cerrada):

$$A_n = 1000 \times (1.00417)^n$$

Proyección:

- Año 1: $1000 \times (1.00417)^{12} \approx \1051.14
- Año 5: $1000 \times (1.00417)^{60} \approx \1283.23
- Año 20: $1000 \times (1.00417)^{240} \approx \2693.71
- Año 50: $1000 \times (1.00417)^{600} \approx \$12,117.77$

Insight: El tiempo es tu mejor aliado en inversiones (exponencial).

D) Torres de Hanoi (Problema Clásico)

Enunciado: Tienes 3 postes y n discos de tamaños diferentes. Objetivo: mover todos de A a C, una regla: nunca un disco grande sobre uno pequeño.

Relación Recursiva:

$$H(n) = H(n-1) + 1 + H(n-1) = 2 \times H(n-1) + 1$$
$$H(1) = 1$$

Desenrolle:

- $H(1) = 1$
- $H(2) = 2(1) + 1 = 3$
- $H(3) = 2(3) + 1 = 7$
- $H(4) = 2(7) + 1 = 15$

Fórmula Cerrada:

$$H(n) = 2^n - 1$$

Verificación: $H(3) = 2^3 - 1 = 7 \checkmark$

Insight Real: Con 64 discos, necesitas $2^{64} - 1 = 18$ billones de movimientos. A 1 movimiento/segundo, ¡tardaría 585 años!

4. SÍNTESIS: Cómo Todo Se Conecta

PROBLEMA REAL

↓

[Modelar con Sucesiones/Recurrencia]

↓

[Encontrar Fórmula Cerrada (Recursividad)]

↓

[Analizar Crecimiento (Lineal vs Exponencial)]

↓

[Probar Validez (Inducción Matemática)]

↓

[Estimar Complejidad (Big O)]

↓

[Implementar Algoritmo Eficiente]

↓

SOLUCIÓN OPTIMIZADA

Ejemplo Completo: Fibonacci Óptimo

✗ Naive (Exponencial): $F(n) = F(n-1) + F(n-2) \rightarrow O(2^n)$

- $F(40)$ tarda ~1 segundo
- $F(50)$ tarda ~17 minutos

✓ Dinámico (Lineal): Almacena resultados previos $\rightarrow O(n)$

- $F(40)$ tarda microsegundos
- $F(50)$ tarda microsegundos

💡 **Insight:** La Matemática Discreta te permite DEMOSTRAR que la solución óptima funciona antes de implementarla.

✅ **AUTOEVALUACIÓN FINAL - Pomodoro 4**

Pregunta 1: Un algoritmo procesa datos con complejidad $O(n^2)$. Si actualmente procesa 10,000 registros en 1 segundo, ¿cuánto tardará con 100,000 registros?

Respuesta esperada:

$O(n^2)$ significa: Tiempo $\propto n^2$

$10,000^2 = 100,000,000$ unidades $\rightarrow$ 1 segundo

$100,000^2 = 10,000,000,000$ unidades

Proporción: $\frac{10,000,000,000}{100,000,000} = 100$

Tiempo: 100 segundos

Pregunta 2: ¿Por qué una SG con $r = 1.01$ eventual supera a una SA con $d = 1000000$?

Respuesta esperada:

SA: $a_n = a_1 + 1000000(n - 1) \rightarrow$ crece linealmente

SG: $b_n = 1 \times 1.01^{n-1} \rightarrow$ crece exponencialmente

En algún n , $1.01^n > 1000000 \times n$

Ejemplo: $n = 2000$

$1.01^{2000} \approx 2.7 \times 10^8$ vs 2×10^9 (SG gana)

📄 **HOJA DE RESUMEN RÁPIDA**

Fórmulas Clave

Concepto	Fórmula	Aplicación
SA	$a_n = a_1 + (n - 1)d$	Ahorros mensuales
SG	$a_n = a_1 \times r^{n-1}$	Interés compuesto
Suma SA	$S_n = \frac{n}{2}(a_1 + a_n)$	Total acumulado lineal
Suma SG	$S_n = a_1 \frac{r^n - 1}{r - 1}$	Total acumulado exponencial
Fibonacci	$F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}}$	Naturaleza y algoritmos
Árbol Binario	$T(h) = 2^{h+1} - 1$	Estructuras de datos

Concepto	Fórmula	Aplicación
Inducción	Base + Paso $\rightarrow$ Verdadero	Pruebas rigurosas

Próximas Acciones

1. **Resuelve los 8 ejercicios de autoevaluación** antes de seguir
2. **Busca ejemplos** de exponenciales en tu vida (redes sociales, consumo de energía, etc.)
3. **Implementa** Fibonacci naive vs dinámico para ver la diferencia
4. **Estudia** relaciones de recurrencia más complejas (como en Rosen, cap. 8)

Referencia: Kenneth H. Rosen - Matemática Discreta y sus Aplicaciones (7ª ed.)

- Capítulo 2: Sucesiones y Sumatorias
- Capítulo 5: Inducción Matemática
- Capítulo 3: Algoritmos

¡Éxito en tu estudio! 